ChatGPT

Executive Summary

We propose a modular, layered architecture for a hackathon-style AI recruitment agent that transforms a *job description (JD)* input into a ranked list of candidate profiles. At a high level, the system consists of a **FastAPI** front-end, an NLP preprocessing layer, an embedding/vector-search layer, a candidate database, a matching/ranking engine, and (optionally) an LLM-based analysis module. A clean separation of concerns (parsers, embedders, stores, scorers) ensures each component can be developed and tested independently. For example, incoming JDs are parsed for keywords and context, embedded into vectors via a **Sentence-Transformers** model [1], and matched against a **vector database** (e.g. FAISS or Weaviate) of candidate embeddings. The top-N candidates are fetched from a relational store (SQLite/Postgres) and scored by a lightweight matching algorithm (e.g. weighted skill/experience scoring). Optionally, a Retrieval-Augmented Generation (RAG) step can refine results: the LLM (open-source or API-based) uses the JD and candidate info to produce explanations or final fit scores [2] [3]. By choosing mature open-source tools (FastAPI, LangChain, Hugging Face models, FAISS/SQLite), we prioritise **rapid prototyping and developer productivity** while still supporting scalability (vector indexing handles millions of items) [4] [1].

This design emphasises *explainability and privacy*. Matching decisions are grounded in traceable data (resumes, skill lists, embeddings) rather than inscrutable black-box models [5] [6]. PII (resume content, contact details) is stored and transmitted securely (TLS, encrypted at rest) and can be anonymised if needed to comply with privacy regulations. Testing will involve unit tests on each module, integration tests of the pipeline, and evaluation of match quality (e.g. precision/recall on known JD–candidate pairs) to ensure accuracy.

Below we detail the architecture in text and diagrams, walk through the data flow step-by-step, break down each module's responsibilities (with data schemas and APIs), recommend specific libraries (with citations), and discuss trade-offs, security/privacy, and testing. All tool recommendations favour open-source or free options (e.g. FastAPI, LangChain, FAISS/Weaviate, SQLite/Postgres, Hugging Face LLMs/embeddings) to enable a fully functional prototype in a limited time.

```
User Interface (e.g. Web GUI or CLI)
    │
    ▼
FastAPI (Web/API Layer)
    │
    ├─► [1] Job Description Parser
    │        • Input: Raw JD text/JSON
    │        • Output: Structured JD data (title, skills, requirements)
    │        • Tech: Regex/spaCy/LLM
    │
    ├─► [2] Embedding Service
    │        • Converts text (JD or resume) to vectors
    │        • Uses Sentence-Transformer model [1]
    │
    ├─► [3] Vector Store (FAISS/Weaviate)
    │        • Stores candidate resume embeddings
```

```
    │        • Provides kNN search by cosine/Euclid distance [3] [7]
    │
    ├─► [4] Candidate Database
    │        • Stores candidate metadata (skills, experience, contact)
    │        • Could be SQLite/Postgres (with pgvector) or simple JSON store
    │
    ├─► [5] Matching & Scoring Engine
    │        • Inputs: JD data + candidate data
    │        • Outputs: Match scores and explanations
    │        • Logic: Weighted skill/expertise matching, optional LLM
refinement
    │
    └─► [6] (Optional) LLM/RAG Module
             • Uses LangChain (or similar) to combine JD + candidate context
             • Produces refined relevance or justification
             • Example: "Given JD and candidate CV, how good is the fit?" [8]

Finally, results (ranked candidates with scores/explanations) are returned as
JSON to the user.
```

```
flowchart LR
    U[User<br/>(Recruiter)]
    API[FastAPI API]
    Parser[JD Parser Module]
    Embed[Embedder Service]
    VDB[Vector DB (FAISS)]
    CandDB[Candidate DB]
    Ranker[Matching & Ranker]
    LLM[LLM/RAG Module]
    UI[Response Formatter]

    U -->|Submit JD| API
    API -->|parse text| Parser
    Parser -->|structured JD| API
    API -->|text embedding| Embed
    Embed -->|query vector| VDB
    VDB -->|top-K IDs| CandDB
    CandDB -->|candidate profiles| Ranker
    Ranker -->|score, rank| API
    API -->|format output| UI
    UI --> U

    %% Optional RAG path
    Ranker -->|LLM query| LLM
    LLM -->|analysis data| Ranker
```

```
sequenceDiagram
    participant U as User
    participant API as FastAPI
```

```
    participant P as JD_Parser
    participant E as Embedder
    participant V as VectorDB
    participant DB as CandidateDB
    participant M as Matcher
    participant L as LLM_Agent

    U->>API: POST /match (job_description)
    API->>P: parse_job_description()
    P-->>API: structured_JD
    API->>E: embed(structured_JD.text)
    E->>V: vector_search(query_vector)
    V-->>DB: get_candidate_ids
    DB-->>M: fetch_candidate_profiles
    M-->>M: compute_initial_scores
    M->>L: (optional) refine_with_LLM(JD, candidate_profiles)
    L-->>M: updated_scores_or_explanations
    M-->>API: ranked_candidates_JSON
    API-->>U: HTTP 200 with results
```

Data Flow (Step-by-Step)

1. **Receive Job Description (JD)**: The user submits a JD (JSON or raw text) via the API. FastAPI validates input using Pydantic schemas.
2. **JD Parsing**: The raw JD text is sent to the *JD Parser* module, which extracts structured fields (e.g. title, skills list, required experience). This might use simple NLP (regular expressions, spaCy NER) or even a small LLM prompt. The output is a `JobDescription` object (see example JSON below).
3. **Embedding the JD**: The parsed JD text (or key sentences) is passed to the *Embedding Service*. We use a Sentence-Transformer (e.g. `all-MiniLM-L6-v2` [9]) to compute a dense vector representing the job's requirements.
4. **Vector Similarity Search**: The JD vector is queried against a **vector database** of candidate embeddings. If using FAISS, we keep all candidate resume embeddings indexed [3] . Alternatively, a hosted DB (Weaviate/Qdrant) can be used. The vector store returns the top-k nearest candidate IDs (by cosine or Euclidean distance).
5. **Candidate Data Retrieval**: Using the candidate IDs, we fetch full profiles from the *Candidate Database* (e.g. SQLite or Postgres with pgvector). Each profile includes metadata: name, skills, years of experience, education, etc.
6. **Initial Matching & Scoring**: The *Matcher* module takes each candidate profile and the JD data to compute a match score. For example, it might score skill overlap (e.g. intersection of JD skills and candidate skills), experience level, or education fit. Weights can be configured (e.g. 40% skills, 30% experience, etc.) [10] .
7. **LLM-Based RAG (Optional)**: For deeper analysis, the system can invoke an LLM (via LangChain/RAG) using both the JD and selected candidate profiles as context. For example, "Judge candidate X's fit for the following JD" with the candidate's CV excerpt. The LLM (open source or API) can output a final score or justification [8] .
8. **Ranking and Output**: Candidates are sorted by combined score. The API formats the results as a JSON array of `{candidate_id, score, highlights/explanation}` objects. This response is returned to the user.

Module Breakdown

1. FastAPI API Layer

- **Responsibilities**: Expose RESTful endpoints (e.g. `/match`) and orchestrate the pipeline. Validate inputs with Pydantic schemas. Handle authentication (if any) and rate limits. Spawn tasks (e.g. background workers) for heavy processing.
- **Inputs/Outputs**: Accepts JSON (JD) or interactive chat messages. Returns JSON responses (ranked candidates, scores).
- **APIs**:
- `POST /match` – body: `JobDescription` JSON, returns `MatchResult` JSON.
- Optional: `POST /upload_candidate` to ingest resume, returning candidate ID.
- **Data Schemas** (Pydantic models):
- `JobDescription`: e.g. `{"job_id":"id","title":"Software Engineer","description":"Develop cloud apps...","requirements":["Python","FastAPI"], ...}`.
- `CandidateProfile`: e.g. `{"candidate_id":"id","name":"Alice","skills":["Python","SQL"],"experience":5,"education":"MSc"}.`
- `MatchScore`: e.g. `{"job_id":"...", "candidate_id":"...", "score":0.87, "details":{...}}`.
- **Error Handling**: Return HTTP 4xx for bad input (invalid JSON/schema) and 5xx for internal errors. Use try/catch and return structured error messages.
- **Scaling**: FastAPI (with Uvicorn/Gunicorn) handles many simultaneous requests. CPU-bound tasks (embedding, vector search) can use asynchronous workers or FastAPI's `BackgroundTasks`. In a prototype, simple threading or uvicorn with multiple workers may suffice.

2. Job Description Parser (NLP Module)

- **Responsibilities**: Extract structured information from a JD. For instance, identify required skills, years of experience, education level. This assists in scoring.
- **Inputs**: Raw text from JD (or JSON fields).
- **Outputs**: Structured JD data (list of skills, keywords, numeric years).
- **Implementation**: Options include keyword extraction (TF-IDF, RAKE), spaCy NER tagging, or a small LLM prompting (e.g. "Extract key skills from the following text"). For hackathon speed, a rule-based or regex approach is easiest.
- **APIs**: Expose a function `parse_jd(text) -> dict`. Could be internal, not an external API.
- **Error Handling**: If parsing fails, fall back to using full text or a default parser. Validate skill lists to be non-empty.

3. Embedding Service

- **Responsibilities**: Convert text (JD or resume/candidate summary) into a numeric vector.
- **Inputs**: Text string (e.g. "Experience with Python, SQL, etc.").
- **Outputs**: Fixed-size embedding (e.g. 384-d vector).
- **Implementation**: Use **Sentence-Transformers** (SBERT) models [1]. For speed, `all-MiniLM-L6-v2` (384d) is compact and fast.
- **APIs**: Internal function `embed(text)`. If using LangChain, use `HuggingFaceEmbeddings`.
- **Error Handling**: Catch OOM errors; use CPU fallback. Cache frequent embeddings if necessary.

4. Vector Database (Similarity Search)

- **Responsibilities**: Store and search high-dimensional vectors efficiently.
- **Inputs/Outputs**: The service receives a query vector and returns nearest-neighbor candidates. Internally, the candidates' vectors are pre-loaded/indexed.
- **Choice of Tool**: Options include **FAISS** (library index), **Weaviate** (DB service), **Qdrant**, or **Postgres with pgvector** [3] [7]. For a prototype, FAISS (open-source library by Meta) is simple and in-memory [3]. Weaviate is more full-featured (adds hybrid search, filters, cloud deployment) [7] but requires running a server.
- **Usage**: On candidate ingestion, compute embeddings and `faiss.IndexFlatIP` (or HNSW) indexes them [3]. At query time, use `index.search(jd_vector, k)`.
- **Error Handling**: Handle "no candidates" gracefully. Tune `k`.
- **Scaling**: FAISS handles millions of vectors with GPU support; for hackathon scale, even thousands fits on RAM [3]. If persistent storage needed, consider storing FAISS index on disk or switching to Weaviate/Milvus.

5. Candidate Database

- **Responsibilities**: Store candidate profiles and metadata. Each record holds structured info (skills, experience, education, resume text if needed, contact info).
- **Implementation**: Could use **SQLite** (easy file-based) or **PostgreSQL**. For vector search, pgvector + Postgres is an option, but with FAISS we just need regular fields.
- **Schema Example (JSON)**:

```
{
  "candidate_id": "1234",
  "name": "Alice Smith",
  "email": "alice@example.com",
  "skills": ["Python", "Machine Learning", "SQL"],
  "experience_years": 5,
  "education": "MSc Computer Science",
  "bio": "Worked on data pipelines...",
  "resume_text": "Alice has experience with Python, AWS, ...",
  "embeddings": [0.12, -0.05, ...]  // (optional, stored in FAISS)
}
```

- **API**: Typically accessed via SQL queries. We might wrap it with an ORM (SQLAlchemy) or Pydantic.
- **Error Handling**: Validate input; on resume upload, handle parse errors.
- **Scaling**: SQLite is fine for ~thousands of profiles. Postgres needed for larger multi-user.

6. Matching & Scoring Engine

- **Responsibilities**: Compute a relevance score between the JD and each candidate, and prepare the ranked output.
- **Inputs**: Parsed JD (skills, requirements), candidate profiles.
- **Outputs**: A list of `{candidate_id, score, details}`.
- **Logic**: A simple approach is weighted sum: e.g. 40% skill overlap score, 30% experience match, 15% education match [10]. For each skill in JD, check if candidate has it; normalize by total skills. Experience score can be min(candidate_exp / jd_exp, 1.0). Education can match levels (PhD/MSc etc). These weights can be adjusted for context.

- **Implementation**: This can be a Python function or class. If using LangChain, you might implement it as a component in a chain. Or simply loop in Python.
- **Optional LLM Step**: For higher sophistication, construct a prompt combining the JD and candidate summary, and ask an LLM: "Rate this candidate's fit from 0–100." The output is parsed and incorporated. LangChain's RAG or ChatChain can manage this [2].
- **APIs**: Internal call `compute_match(jd, candidate)` returns score and rationale.
- **Error Handling**: If any field is missing, fallback to partial scoring. If LLM API fails, skip or use default.
- **Scaling**: Ranking hundreds of candidates is fast; if thousands, parallelize the loop or use vectorized ops.

7. LLM/RAG Module (Optional)

- **Responsibilities**: Use a Language Model (GPT or open equivalent) to interpret job and candidate texts for nuanced matching or explanation.
- **Use Cases**: Generating human-like explanations: "Why is candidate X a good fit?", or extracting implicit skills from a resume.
- **Implementation**: Via **LangChain** chains or a simpler call. For example, feed the top-5 candidate resumes plus the JD into a RetrievalQA chain with a prompt template. The assistant can output scores or summaries [8].
- **Tools**: Could use Hugging Face models (e.g. LLaMA2, Mistral, or Meta's open Llama) or APIs (OpenAI, Anthropic). For hackathon, free local models (Llama2-13B) or smaller (Vicuna) might be preferable for quick demos. LangChain supports many LLM providers.
- **Input**: Text context (JD + candidate text) and a question/prompt.
- **Output**: Typically text; use `StrOutputParser()` to get a string [11].
- **Error Handling**: Manage rate limits. Keep temperature=0 for deterministic scoring. Validate output format.

8. User Interface / Conversational Layer

- **Responsibilities**: (Optional) If an interactive chat/bot is needed, the system can be wrapped with a chatbot interface (e.g. via Streamlit, Gradio, or Slack/Teams bot). This can allow recruiters to ask follow-up questions (e.g. "Show me candidates with Python experience").
- **Implementation**:
- **UI App**: A simple web page (HTML/JS) or a Streamlit/Gradio app. For hackathon speed, Streamlit or a lightweight React + FastAPI backend.
- **Chatbot**: Use **LangChain's** chat interfaces or **Rasa**/Bot frameworks if needed. However, for MVP, returning JSON is sufficient.
- **Scaling**: UI frames and chat are beyond core; can be minimal.

9. CI/CD and DevOps

- **Tools**: Use **Docker** to containerize the app (ensures same env, easier deployment) and **GitHub Actions** for CI (linting, testing). Tests (pytest) should cover key functions (parsing, embedding, matching).
- **Monitoring**: Logs via standard logging. For production readiness: Prometheus/Grafana for metrics, Sentry/Loguru for error tracking. In a hackathon, simple logs and script health checks suffice.
- **Deployment**: The app can run on any Linux server (no cloud-specific reliance). SQLite means no external DB setup; optionally Alpine Linux image.

Example Data Schemas

Below are representative JSON schemas for the main data entities. These would be implemented as Pydantic models in code.

Job Description (`JobDescription`):

```
{
  "job_id": "job-001",
  "title": "Software Engineer",
  "description": "Develop scalable web applications in Python",
  "requirements": ["Python", "FastAPI", "SQL", "Docker"],
  "min_experience_years": 3,
  "education": "Bachelor",
  "location": "London"
}
```

Candidate Profile (`CandidateProfile`):

```
{
  "candidate_id": "cand-123",
  "name": "Alice Smith",
  "email": "alice@example.com",
  "skills": ["Python", "Django", "AWS", "SQL"],
  "experience_years": 4,
  "education": "MSc Computer Science",
  "bio": "4 years as backend engineer at FinTech startup.",
  "resume_text": "Alice has 4 years experience with Python, Django, and
AWS.",
  "contact_info": {"phone": "+44...", "linkedin": "linkedin.com/in/alice"}
}
```

Match Score (`MatchScore`):

```
{
  "job_id": "job-001",
  "candidate_id": "cand-123",
  "score": 0.85,
  "details": {
    "skill_overlap": 0.75,
    "experience_score": 0.80,
    "education_match": 1.0,
    "explanation": "Candidate has most required skills, suitable experience."
  }
}
```

Pydantic can enforce types and ranges (e.g. `score: float = Field(ge=0, le=1)`).

Tools, Libraries, and Implementation Tips

For rapid prototyping with Python:

- **FastAPI** (API layer) – a modern Python web framework, high-performance (async), auto-generates docs. Official tutorial：https://fastapi.tiangolo.com/
- **LangChain** (or **LlamaIndex**) – framework for chaining LLM calls and retrieval. LangChain has built-in support for embeddings and vector stores [12] [13]. Use it to glue together embedding, vector DB, and LLM steps.
- **Sentence-Transformers** (Hugging Face) – use models like `"sentence-transformers/all-MiniLM-L6-v2"` [9] for embeddings. Very fast and state-of-art for semantic search.
- **Vector DB**:
- **FAISS** – runs locally, zero config, very fast (supports HNSW/IVF) [3]. Great for in-memory prototyping.
- **Weaviate** – open-source vector DB with REST API, has built-in RAG support and filters [7]. Useful if you want a DB container (but heavier to set up).
- **pgvector (Postgres)** – if using Postgres anyway, a simple alternative.
- **Database**: **SQLite** (file-based) or **PostgreSQL**. SQLite is easiest for single-user demos; Postgres scales to multi-user and can host pgvector.
- **Document Parsing**: To ingest resumes, use libraries: PyPDF2/pdfplumber (PDF), python-docx, python-pptx [14].
- **Chat/Agent Frameworks**: If building a conversational agent interface, consider **LangChain's ConversationalRetrievalQA**, **OpenAI's Functions** (if using API), or simple Bot frameworks like Rasa (though Rasa has steep setup). For hackathon, a Q&A pattern via FastAPI (user asks "find candidates with X") is simpler.
- **CI/CD**: GitHub Actions (workflows for testing and linting). Tools like `pytest`, `flake8`, `mypy`. Use Docker Compose to spin up FAISS/Weaviate for integration tests.
- **Monitoring**: In production, consider Prometheus instrumentation (`prometheus-client`) and Grafana; for a hackathon prototype, simple logging (Python `logging`) is acceptable.
- **LLM Model Choice**: For on-device inference, use open models: LLaMA-2 (Meta), Mistral 7B, or open variants from Hugging Face. For example, Mistral-7B is available for inference and fast. If using an API, GPT-3.5-turbo (via OpenAI) or Claude (Anthropic) can give high-quality results but incur cost.
- **RAG Tools**: LangChain for orchestrating retrieval + generation. LangChain's `RetrievalQA` or custom chains can be used. Alternatively, **Haystack** by deepset is a popular open-source QA framework that includes RAG patterns.
- **Quick-start Templates**: Many GitHub repos (e.g. RAG-with-Langchain-and-FastAPI [15]) show how to combine these pieces. The RAG Recruitment Assistant (anghelomp10) demonstrates CV processing with LangChain [16] [17].

Trade-offs

Component	Options (Pros)	Cons/Trade-offs
LLM	OpenAI GPT-3.5/GPT-4 (best quality) [18]	Closed-source, cost per token, dependent on API
	Llama-2, Mistral, Vicuna (local; free)	Limited by GPU RAM, possibly lower quality

Component	Options (Pros)	Cons/Trade-offs
Vector Store	FAISS (local, blazing-fast) [3]	In-memory (needs RAM), no out-of-the-box filtering
	Weaviate/Milvus (full DB, hybrid search)	More complex to deploy, requires container/cluster
	PostgreSQL + pgvector (single integrated DB)	Slower at scale, requires tuning
Embeddings	Sentence-Transformers (open, many models) [1]	Need to manage model downloads, CPU/GPU requirements
	OpenAI Embeddings (e.g. Ada)	API cost, data sent to cloud, rate limits
Web Framework	FastAPI (async, easy, great docs)	Python-only (for hackathon, fine)
	Flask (widespread, simple)	Less modern features (no async by default)
Data Storage	SQLite (file, zero-config)	Single-writer, limited concurrency
	PostgreSQL (robust, supports concurrency)	Need to run DB server/container
RAG Tooling	LangChain (flexible chains)	Learning curve; adds abstraction layer
	Weaviate RAG (built-in) [7]	Vendor lock-in to Weaviate features
Deployment	Local Python app (easy)	Harder to scale horizontally; no multi-node
	Docker/Kubernetes (scalable)	More complex setup; overhead

Note: For a hackathon, favor simplicity: use local, easy-to-start tools (FastAPI, SQLite, FAISS) [3] [1]. Cloud services (AWS Lambda, Azure Functions) could be used but not required.

Security and Privacy Considerations

- **Data Protection**: Resumes and job descriptions contain personal data. Use HTTPS (TLS) for all API calls. Store sensitive info encrypted at rest (disk encryption for SQLite, or use DB features).
- **PII Handling**: Strip or hash PII when not needed (e.g. only store contact email if needed for contacting). Comply with GDPR by allowing data deletion. Do not send raw CV text to external APIs unless consented.
- **Access Control**: If multiple users, implement authentication (API keys or OAuth) on FastAPI. Limit endpoints (e.g. only HR staff can retrieve candidate emails).
- **Model Privacy**: If using cloud LLM APIs, redact confidential info or use on-premise models to keep data in-house.
- **Audit Trail**: Log matching decisions (what criteria led to recommendation) to ensure explainability [5]. This aids compliance and debugging.
- **Bias Mitigation**: Ensure scoring focuses on job-relevant skills, not protected attributes. For a prototype, note this as a caveat.

Testing and Validation

- **Unit Tests**: Write tests for each module: JD parser extracts skills correctly, embedding outputs expected shapes, matching logic scores known examples. Use `pytest`.
- **Integration Tests**: Simulate full flow with sample JDs and a small candidate dataset. Verify the top-ranked candidate matches intuitive expectations.
- **Performance Tests**: Measure inference latency for embedding and LLM; ensure API stays responsive. Use smaller models if too slow.
- **Data Validation**: Use Pydantic models to enforce schemas. Validate that input JD JSON has all required fields, and candidate data has no null skills.
- **Quality Evaluation**: If labeled data is available, compute accuracy of matches. Otherwise, do manual review by domain experts.
- **Monitoring**: In prod mode, track metrics (requests per second, average score, error rates). For prototype, console logs suffice.

By following this design, a hackathon team can quickly build a functional recruitment assistant that ingests job descriptions, semantically searches a candidate pool, and outputs ranked matches with clear reasoning. The modular clean architecture ensures each piece can be improved or swapped (e.g. upgrading to a more powerful LLM) without rewriting the whole system [19] [5]. The use of open-source tools and lightweight frameworks (FastAPI, LangChain, FAISS, SQLite) keeps the stack manageable for rapid development and iteration.

Sources: Established best practices in RAG architecture [20] [21], open-source libraries (Sentence-Transformers [1], FAISS [3], Weaviate [7]), and industry guidelines on AI explainability and ethics [5] were referenced in crafting this design.

---

[1] [9] SentenceTransformers Documentation — Sentence Transformers documentation
https://sbert.net/

[2] [4] [6] [8] [11] [12] [13] [16] [17] [18] [19] [20] [21] RAG Architecture - RAG Recruitment Assistant
https://anghelomp10-rag-recruitment-assistant.mintlify.app/concepts/rag-architecture

[3] [7] Best open source vector database software: Top 8 in 2026
https://www.instaclustr.com/education/vector-database/best-open-source-vector-database-software-top-8-in-2026/

[5] AI Recruiting in 2026: The Definitive Guide
https://www.phenom.com/blog/recruiting-ai-guide

[10] [14] Building an AI-Powered Talent Match System with Multi-Agent Architecture
https://www.linkedin.com/pulse/building-ai-powered-talent-match-system-multi-agent-ravi-prabakaran-t7aoe

[15] anarojoecheburua/RAG-with-Langchain-and-FastAPI: This ... - GitHub
https://github.com/anarojoecheburua/RAG-with-Langchain-and-FastAPI